Dijiste:

CRITICAL SUBMISSION POLICY
Source Whitelist and Quota Killswitch: Live ASR capture is restricted to the instructor whitelist and the instructor RTMP demo stream (INV-10); any other URL fails the ingest precondition. API usage is bounded by your team quota; exhaustion triggers an automatic fallback to the CPU baseline or free tier (INV-18) and carries no emergency bailout.
Cadence Notice: Released Fri 09 Oct 2026 (Wildcat Pause Day); the Tue 20 Oct 2026 due date provides an 11-day window in observance of the pause day. MP 5 is due 5 days before PS 6 (Sun 25 Oct 2026), an explicit exception to the standard 6-day inter-assignment gap, applied to maintain assignment cadence without forcing a due date on Halloween (Sat 31 Oct).
Individual Assignment
Data Analytics Pipelines (Structured Streaming)
Event-time windowed analytics, checkpointed fault tolerance, and contract-validated ingest over replayed and live sources
📅 Assigned:
Fri 09 Oct 2026
⏰ Due:
Tue 20 Oct 2026
💯 Points:
30
📂 Submit:
Git tag mp5-submission
Required Readings
The Dataflow Model
Akidau et al., VLDB 2015: Event time vs. processing time; windows, watermarks, and triggers; completeness, latency, and cost trade-offs.
Pipeline Context: Upstream inputs: MP 3 landing zone; Downstream consumers: PS 6, MP 7, PS 8, HW 12.
📖 Access Resource →
Structured Streaming Programming Guide
Spark 3.5.7: Programming model through fault tolerance: window operations, late data and watermarking, output modes, checkpointing, exactly-once semantics.
📘 Access Resource →
Part 1: Initiating Your Draft (The AI Kit Procedure)
(Ungraded Preparation Step)

Use the Generative AI Kit to help structure your submission. Follow these stages to systematically break down your assignment:
Execution Steps
Contract Extraction: Have the AI Kit derive the record StructType and deal contracts (BoundedChunking: chunk_duration_sec <= 30, ts_start < ts_end; whitelist and provenance gates) from this specification, then verify every nullability choice yourself.
Pipeline Scaffold: Generate the readStream / withWatermark / groupBy(window(...)) skeleton with an explicit schema; audit it for schema inference, missing watermarks, unbounded state, and non-idempotent sinks.
Event-Time Calibration: Replay the landing zone, measure the lateness distribution ingest_ts - ts_end (p50/p95/p99), and derive the watermark delay from it.
Fault Injection: Set checkpointLocation, write to an append-mode Parquet sink, kill the query mid-run, restart it, and reconcile row counts and window keys.
Run Report: Assemble metrics.jsonl and the report sections; verify that every reported number reproduces on the CPU baseline.
Part 2: Required Document Structure
(30 Points)
Objective: Your submission must be a single report.md (at most 3 pages) delivered with Git tag mp5-submission (along with your code, contracts.py, metrics.jsonl, and reconciliation notebook) containing the following sections exactly as named below.
Schema and Contracts: Explicit StructType (TranscriptChunk for ASR; CorpusChunk for VDB: record_id, track, source_id, ts_start, ts_end, text, ingest_ts, plus license_class for corpus records), nullability justification, and the deal contracts.
Watermark and State Analysis: Lateness percentiles, the chosen watermark delay and its completeness-vs-latency trade-off, the required output mode, and the state-growth bound.
Query Design: 60-second tumbling and 5-minute/1-minute sliding windows computing record count, approx_count_distinct of tokens, top-10 terms, and mean ingest latency; sink design.
Fault-Tolerance Reconciliation: Restart procedure; row-count and window-key reconciliation showing no duplicate or missing windows; which exactly-once guarantees come from checkpointing versus an idempotent sink.
Metrics and Reproducibility: StreamingQueryProgress summary (inputRowsPerSecond, processedRowsPerSecond, batchDuration, stateOperators.numRowsTotal), pinned-environment statement, CI

status, and the AssemblyAI quota log (ASR track). Throughput and latency targets are liveness properties (INV-5) assessed from the metrics log, not from contracts.
GenAI Audit: Did you use the Generative AI Kit to synthesize this artifact? You must
Deliverable: All work must be solvable on the pinned CPU DevContainer (INV-14/INV-15); GPU access is optional and never required.
Scoring Rubric (30 Points Total)
Component Weight Description
Part 1: Contracts & Event-Time Analysis 9 pts Schema & contracts (4.5 pts): Explicit schema, justified nullability, bounded chunking/provenance gates.
Watermark & state analysis (4.5 pts): Watermark derived from measured lateness; output mode and state bound justified.
Part 2: Streaming Pipeline 19.5 pts Query correctness (7.5 pts): Window definitions and aggregates are correct on reference replay.
Fault tolerance (7.5 pts): Restart demonstration with exact reconciliation and no duplicate window keys.
Metrics & reproducibility (4.5 pts): Metrics log complete; pinned environment; CI green on the CPU baseline.

WHAT DO I HAVE TO DO?

ChatGPT dijo:

Dijiste:

CIS731.zip
Archivo

cis531-731-2026...
Archivo

CAN I USE ANY OF THESE FILES?

Procesó durante 18s

ChatGPT dijo:

Dijiste:

labs.zip
Archivo

THIS WAS MP3

ChatGPT dijo:

Dijiste:

YES, THAT WAS I SUBMIT

Minimum Quantum of Work (Execution Floor)

This assignment establishes an immutable baseline of execution. You must successfully execute the core regex modification in PySpark and successfully ingest a transcription into your local FAISS database. Do not reduce the scope of the problem if you encounter environment friction; resolve the friction using your DevContainer.
Machine Problem

MP 3: Real-Time ASR & Vector Database Ingestion

Whisper Transcriptions, FAISS Embeddings, & Trope Ontologies
📅 **Assigned:**
Sat 19 Sep 2026
⏰ **Due:**
Mon 28 Sep 2026, 11:59 PM
Sun 04 Oct 2026, 11:59 PM
💯 **Points:**
30
📂 **Submit:**
GitHub PR & Canvas link

Required Laboratory Notebooks

Repository SSOT: github.com/kddresearch/cis531-731-2026_fall

Lab 2: The "Scrub" Stage

Data Cleaning, Text Tokenization & TF-IDF Feature Extraction in PySpark.
View on GitHub → Canvas Preview →

Lab 3a: Local ASR

Initializing local OpenAI Whisper for audio stream transcriptions.
View on GitHub → Canvas Preview →

Lab 3b: VDB Acquisition

Generating latent embeddings via HuggingFace and indexing in FAISS.
View on GitHub → Canvas Preview →

Part 1: The Scrub Pipeline & Hyphenation Parsing

(15 Points)
In data science, sloppy text normalization destroys semantic relationships downstream. You must utilize PySpark SQL functions to clean a dirty dataset while retaining critical domain-specific syntax.

Tasks:

- Execute the entirety of the **Lab 2** PySpark notebook (`lab02.ipynb`) in your local or containerized SparkSession.
- **Modification Requirement:** Edit the `regexp_replace` function in Part 2 of the notebook. The default regex strips all special characters. You must update the regex string to specifically **retain hyphenated compound words** (e.g., ensuring `large-scale` remains a single token rather than becoming `largescale` or splitting into `large` and `scale`).
- Generate and print the final TF-IDF sparse vector representation containing your correctly hyphenated tokens.

Part 2: ASR to VDB Integration (The "Obtain" Stage)

(15 Points)

Objective: Prove end-to-end integration by taking raw audio, converting it to text via a local model, mapping it to latent space, and retrieving it via semantic similarity.

1. Execute **Lab 3a** to download the public-domain WWI Poetry archive (.zip), extract Wilfred Owen's "Anthem for Doomed Youth," and load the Whisper `base` model to successfully transcribe the audio.
2. Execute **Lab 3b** to initialize the `all-MiniLM-L6-v2` embedder and the FAISS index.
3. **Integration Task:** Take the exact `result["text"]` variable generated by your Whisper model in Lab 3a, and paste it into the designated integration block in Lab 3b so it is embedded alongside the existing mock corpus.
4. Execute a semantic query searching for the concept `"doomed youth"` and verify that it retrieves the LibriVox transcript.
5. **CIS 731 (Graduate) Extension:** Instead of hardcoding the FAISS index in memory, write a function that saves the FAISS index to disk (`faiss.write_index`) after ingestion, then loads it back into memory to perform the query.

Deliverable: Commit your completed Jupyter Notebooks (`.ipynb` files) containing the executed cell outputs for both Part 1 and Part 2. Submit the link to your GitHub Pull Request on Canvas.

Scoring Rubric

Component	Weight	Description
PySpark Scrub & Regex	15 pts	Successfully executed the PySpark TF-IDF pipeline with the regex correctly modified to retain hyphenated compound words in the final token array.

Component	Weight	Description
ASR & VDB Integration	15 pts	Successfully transcribed the LibriVox audio via Whisper, embedded the output, ingested it into FAISS, and executed a semantic trope query. (731 students graded on disk I/O implementation).

🏠 Return to Course Home
📝 All HW⬆️ Module⏫ Phase

	File Name	Size
svg	labs.zip	20.8 KB

ChatGPT dijo:

Dijiste:

SO TECNICALLY, WHICH FILES I WOULD NEED TO CREATE

ChatGPT dijo:

Dijiste:

streaming_pipeline.py
Most work

reconciliation.ipynb
Significant testing

contracts.py
Moderate

report.md
Writing

metrics.json

this are the files i need to create?

ChatGPT dijo:

Dijiste:

can you give a small definiton of each one in simple english and what would be the function?

ChatGPT dijo:

Dijiste:

all code syntax in python i will need to complete this assigmemt for each file

ChatGPT dijo:

Dijiste:

since the pass mp3 did not have specifications did not contain an identifiable timestamped JSON/Parquet landing-zone dataset suitable for MP 5. what should i do?

ChatGPT dijo:

Dijiste:

how i submit this logs for the genAi part

ChatGPT dijo:

Dijiste:

Skip To Content
Genesis Joseph
Account
Dashboard
Courses
Groups
Calendar
30
Inbox
History
10
Help
CIS 731 A
Assignments
[MP] Homework 05 of 12 (Machine Problem): Structur...

Fall 2026
Home
IgniteAI Search
Announcements
Assignments
Discussions

Grades
People
Pages
Files
Course Schedule
BigBlueButton
Collaborations
Library Resources
Syllabus
Microsoft Education
Notebook
[MP] Homework 05 of 12 (Machine Problem): Structured Streaming
Due: Tue Oct 20, 2026 11:59pm
Attempt 1
In Progress
NEXT UP: Submit Assignment

Unlimited Attempts Allowed
Home » Phase 2: Obtain & Ingest » Module 04: Data Analytics Pipelines (Structured Streaming) » MP 5
All Assignments
CRITICAL SUBMISSION POLICY
Source Whitelist and Quota Killswitch: Live ASR capture is restricted to the instructor whitelist and the instructor RTMP demo stream (INV-10); any other URL fails the ingest precondition. API usage is bounded by your team quota; exhaustion triggers an automatic fallback to the CPU baseline or free tier (INV-18) and carries no emergency bailout.

Cadence Notice: Released Fri 09 Oct 2026 (Wildcat Pause Day); the Tue 20 Oct 2026 due date provides an 11-day window in observance of the pause day. MP 5 is due 5 days before PS 6 (Sun 25 Oct 2026), an explicit exception to the standard 6-day inter-assignment gap, applied to maintain assignment cadence without forcing a due date on Halloween (Sat 31 Oct).

Individual Assignment
Data Analytics Pipelines (Structured Streaming)
Event-time windowed analytics, checkpointed fault tolerance, and contract-validated ingest over replayed and live sources

Assigned:

Fri 09 Oct 2026
⏰ Due:
Tue 20 Oct 2026
💯 Points:
30
📂 Submit:
Git tag mp5-submission
Required Readings
The Dataflow Model
Akidau et al., VLDB 2015: Event time vs. processing time; windows, watermarks, and triggers; completeness, latency, and cost trade-offs.
Pipeline Context: Upstream inputs: MP 3 landing zone; Downstream consumers: PS 6, MP 7, PS 8, HW 12.
📖 Access Resource →
Structured Streaming Programming Guide
Spark 3.5.7: Programming model through fault tolerance: window operations, late data and watermarking, output modes, checkpointing, exactly-once semantics.
📘 Access Resource →
Part 1: Initiating Your Draft (The AI Kit Procedure)
(Ungraded Preparation Step)

Use the Generative AI Kit to help structure your submission. Follow these stages to systematically break down your assignment:

Execution Steps
Contract Extraction: Have the AI Kit derive the record StructType and deal contracts (BoundedChunking: chunk_duration_sec <= 30, ts_start < ts_end; whitelist and provenance gates) from this specification, then verify every nullability choice yourself.
Pipeline Scaffold: Generate the readStream / withWatermark / groupBy(window(...)) skeleton with an explicit schema; audit it for schema inference, missing watermarks, unbounded state, and non-idempotent sinks.
Event-Time Calibration: Replay the landing zone, measure the lateness distribution ingest_ts - ts_end (p50/p95/p99), and derive the watermark delay from it.
Fault Injection: Set checkpointLocation, write to an append-mode Parquet sink, kill the query mid-run, restart it, and reconcile row counts and window keys.
Run Report: Assemble metrics.jsonl and the report sections; verify

that every reported number reproduces on the CPU baseline.
Part 2: Required Document Structure
(30 Points)

Objective: Your submission must be a single report.md (at most 3 pages) delivered with Git tag mp5-submission (along with your code, contracts.py, metrics.jsonl, and reconciliation notebook) containing the following sections exactly as named below.

Schema and Contracts: Explicit StructType (TranscriptChunk for ASR; CorpusChunk for VDB: record_id, track, source_id, ts_start, ts_end, text, ingest_ts, plus license_class for corpus records), nullability justification, and the deal contracts.
Watermark and State Analysis: Lateness percentiles, the chosen watermark delay and its completeness-vs-latency trade-off, the required output mode, and the state-growth bound.
Query Design: 60-second tumbling and 5-minute/1-minute sliding windows computing record count, approx_count_distinct of tokens, top-10 terms, and mean ingest latency; sink design.
Fault-Tolerance Reconciliation: Restart procedure; row-count and window-key reconciliation showing no duplicate or missing windows; which exactly-once guarantees come from checkpointing versus an idempotent sink.
Metrics and Reproducibility: StreamingQueryProgress summary (inputRowsPerSecond, processedRowsPerSecond, batchDuration, stateOperators.numRowsTotal), pinned-environment statement, CI status, and the AssemblyAI quota log (ASR track). Throughput and latency targets are liveness properties (INV-5) assessed from the metrics log, not from contracts.
GenAI Audit: Did you use the Generative AI Kit to synthesize this artifact? You must include your GenAI.pdf log or explicitly state "No GenAI Used."
Deliverable: All work must be solvable on the pinned CPU DevContainer (INV-14/INV-15); GPU access is optional and never required.

Scoring Rubric (30 Points Total)
Component Weight Description
Part 1: Contracts & Event-Time Analysis 9 pts Schema & contracts (4.5 pts): Explicit schema, justified nullability, bounded chunking/provenance gates.
Watermark & state analysis (4.5 pts): Watermark derived from

measured lateness; output mode and state bound justified.
Part 2: Streaming Pipeline 19.5 pts Query correctness (7.5 pts): Window definitions and aggregates are correct on reference replay.
Fault tolerance (7.5 pts): Restart demonstration with exact reconciliation and no duplicate window keys.
Metrics & reproducibility (4.5 pts): Metrics log complete; pinned environment; CI green on the CPU baseline.
GenAI Verification 1.5 pts Includes the required GenAI log (GenAI.pdf) or explicitly states AI was not used.
🏠 Return to Course Home
📝
All HW
⬆️
Module
⏫
Phase
Choose a submission type

ChatGPT dijo: